

۱. هدف سند

این سند استانداردهای معماری و توسعه Frontend سیستم ERP SaaS را تعریف می‌کند.
هدف:

- ایجاد رابط کاربری مدرن، سریع و قابل توسعه
- پشتیبانی همزمان از Web Application و Mobile Application
- ایجاد تجربه کاربری یکپارچه برای مشتریان مختلف SaaS
- جلوگیری از پیچیدگی و کدهای غیرقابل نگهداری در توسعه بلندمدت
- هماهنگی کامل با معماری Backend و API First

۲. تکنولوژی‌های اصلی Frontend

Web Application

تکنولوژی اصلی:

- Next.js
- React
- TypeScript

استفاده از JavaScript خام در پروژه ممنوع است و تمامی کدهای Frontend باید با TypeScript توسعه داده شوند.

۳. اصول معماری Frontend

معماری Frontend بر اساس اصول زیر طراحی می‌شود:

Component Based Architecture

تمام رابط کاربری باید بر اساس Component های مستقل و قابل استفاده مجدد ساخته شود.
قوانین:

- Component ها باید کوچک و قابل نگهداری باشند.
- Logic کسب‌وکار نباید داخل Component های UI قرار گیرد.
- Component های عمومی باید در Shared Layer قرار گیرند.

Feature Based Architecture

ساختار پروژه باید بر اساس Feature ها و ماژول‌های کسب‌وکار تقسیم شود، نه صرفاً نوع فایل.
هر ماژول ERP باید محدوده مستقل خود را داشته باشد.

۴. ساختار استاندارد پروژه Frontend

ساختار پیشنهادی:

src

```
├── app
|
├── modules
|   |
|   ├── module-name
|   |   ├── components
|   |   ├── hooks
|   |   ├── services
|   |   ├── types
|   |   └── validations
|   └── pages
|
├── shared
|   ├── components
|   ├── layouts
|   ├── hooks
|   ├── utils
|   ├── constants
|   └── types
|
├── api
|
├── auth
|
├── store
|
└── styles
```

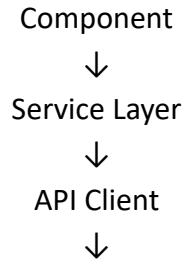
قانون ارتباط با Backend

Frontend فقط از طریق API با Backend ارتباط دارد.

قوانین:

- دسترسی مستقیم به Database ممنوع است.
- Component ها نباید مستقیماً API Call کنند.
- تمام ارتباطات باید از Service Layer عبور کنند.

ساختار:



Backend API

۶. استاندارد API Client

تمام ارتباطات API باید از یک لایه مرکزی مدیریت شود.

مسئولیت: API Client

- مدیریت Token
- مدیریت Header ها
- مدیریت Error ها
- مدیریت Refresh Token
- استاندارد سازی Request و Response

۷. مدیریت State

State Management به دو بخش تقسیم می شود:

Server State

برای داده های دریافت شده از Backend:

Technology:

TanStack Query

مسئولیت:

- دریافت داده
- Cache
- Synchronization
- Loading State
- Error Handling

Client State

برای وضعیت داخلی برنامه:

Technology:

Zustand

موارد استفاده:

- وضعیت UI
- تنظیمات کاربر
- Tenant Context
- وضعیت Session

۸. مدیریت فرم‌ها

تمام فرم‌های سیستم باید استاندارد مشخص داشته باشند.

Technology:

- React Hook Form
- Zod Validation

قوانین:

- Validation باید مشخص و قابل تست باشد.
- قوانین اعتبارسنجی نباید فقط در Frontend باشد.
- Backend همیشه مرجع نهایی Validation است.

۹. استاندارد UI و Design System

Technology پیشنهادی:

- Tailwind CSS
- Shadcn/UI

هدف:

- ایجاد Design System یکپارچه
- جلوگیری از طراحی‌های متفاوت در ماژول‌ها
- افزایش سرعت توسعه

۱۰. Responsive Design

تمام صفحات باید با رویکرد Mobile First طراحی شوند.

اولویت طراحی:

Mobile



Tablet



Desktop

هیچ صفحه‌ای نباید فقط برای Desktop طراحی شود.

۱۱. معماری Mobile Application

نسخه موبایل با تکنولوژی:

React Native

توسعه داده می‌شود.

هدف:

- اشتراک دانش بین Web و Mobile
- کاهش هزینه توسعه
- هماهنگی تجربه کاربری

۱۲. مدیریت Authentication

Frontend مسئول Authentication نیست و فقط مصرف‌کننده سرویس Backend است.

وظایف: Frontend:

- نگهداری Session
- مدیریت وضعیت Login
- نمایش وضعیت کاربر

وظایف: Backend:

- اعتبارسنجی هویت
- صدور Token
- کنترل امنیت

۱۳. مدیریت Permission و Authorization

Backend مرجع اصلی تصمیم‌گیری دسترسی است.

Frontend فقط:

- کنترل نمایش UI
- مخفی کردن گزینه‌های غیرمجاز
- بهبود تجربه کاربری

را انجام می‌دهد.

هیچ Permission مهمی نباید فقط در Frontend کنترل شود.

۱۴. مدیریت Multi Tenant در Frontend

Frontend باید Tenant Context داشته باشد.

اطلاعات Tenant شامل:

- tenant_id
- tenant_name
- فعال بودن ماژول‌ها
- Permission های کاربر

است.

تمام درخواست‌های API باید Context مربوط به Tenant را داشته باشند.

۱۵. مدیریت Feature Access و Module

Frontend باید قبل از نمایش بخش‌های سیستم بررسی کند:

- آیا Tenant این Module را فعال دارد؟
- آیا User Permission لازم را دارد؟

اما تصمیم نهایی همیشه در Backend انجام می‌شود.

۱۶. Performance Rules

قوانین:

- جلوگیری از Render غیر ضروری
- استفاده صحیح از Cache
- Lazy Loading برای بخش‌های سنگین
- بهینه‌سازی تصاویر و Asset ها
- مدیریت مناسب Bundle Size

۱۷. Error Handling

تمام خطاهای API باید استاندارد باشند.

Frontend باید:

- پیام مناسب کاربر نمایش دهد.
- خطاهای فنی را Log کند.
- تجربه کاربری مناسب ایجاد کند.

۱۸. Logging و Monitoring

Frontend باید قابلیت ثبت:

- خطاهای JavaScript
- خطاهای API
- مشکلات Performance

را داشته باشد.

۱۹. استاندارد توسعه کد

قوانین:

- TypeScript اجباری است.
- Naming Convention پروژه باید رعایت شود.
- Component های تکراری ممنوع هستند.
- Business Logic نباید داخل UI Component باشد.
- Code Review قبل از Merge الزامی است.

۲۰. اصل نهایی معماری Frontend

هر تصمیم توسعه‌ای باید این سوال را پاسخ دهد:

"آیا این تصمیم باعث افزایش قابلیت توسعه، نگهداری آسان‌تر و تجربه بهتر کاربر در بلندمدت می‌شود؟"

اگر پاسخ منفی باشد، آن تصمیم نباید وارد معماری اصلی شود.

نتیجه نهایی

Frontend سیستم ERP SaaS بر اساس Stack زیر توسعه خواهد یافت:

Next.js

+

React
+
TypeScript
+
Tailwind CSS
+
Shadcn/UI
+
TanStack Query
+
Zustand
+
React Hook Form
+
Zod
+
React Native

این سند مرجع اصلی تصمیمات معماری Frontend پروژه است و تمامی توسعه‌های آینده باید مطابق آن انجام شود.